KHAGAN LIVE BAKERY

Bakery Management ERP + POS

Google Antigravity • Full Development Master Plan • Version 1.0 • October 2026

Implementation-ready product requirements, database design, bilingual interface specification, admin and shop portals, inventory and costing rules, security controls, phased delivery, and acceptance tests.

Executive build directive

Build a real, production-oriented, bilingual (Bangla + English) bakery management web app with two independent role-protected interfaces: Admin at /admin and Shop POS at /shop. Both must use one central PostgreSQL database and share verified transactions. Do not deliver a static UI or mock-data demo as the finished system.

Business locale: Bangladesh; currency: BDT (□); timezone: Asia/Dhaka. Default UI language: Bangla. Prioritize ease of use, accurate inventory and financial accounting, security, and fast shop checkout.

1. Scope and portals

Admin / Management portal	Shop / Cashier portal
Owner dashboard, recipes, production planning, material stock, finished stock, purchases, supplier balances, pricing, expenses, utility allocation, workers, payroll, profit, reports, user permissions and audit logs.	Fast POS, product search and categories, cart, cash/mobile payments, split payments, receipt print, shop stock, authorized returns, shop waste, incoming transfers, shift opening/closing and cash reconciliation.

2. Technology and architecture

- Next.js App Router + TypeScript; Tailwind CSS + shadcn/ui; responsive PWA; React Hook Form + Zod; TanStack Query; Recharts.
- Supabase PostgreSQL + Auth + Row Level Security; server-side functions/transactional RPC for critical stock and financial posting.
- next-intl or equivalent: Bangla and English dictionaries, translated labels, validation, receipts, reports; persist user language preference.
- Deploy on Vercel; configure secure environment variables; maintain SQL migrations, .env.example, README, test suite, and backup/recovery plan.
- Multi-branch-ready organization_id, branch_id, location_id in relevant records; initially launch one branch.

3. UX / visual system

- Premium white + forest-green UI: primary #176B45, dark #104C34, light #EAF5EE, background #F7F9F8, text #172B22, danger #D94B4B.
- Admin desktop-first: sidebar, top search/date filters, dashboard KPIs, clear tables, forms, reports, alerts and approval queues.
- Shop touch-first: large product cards, search, category tabs, cart at right on desktop, single clear checkout action, numeric keypad-friendly input.
- Use Noto Sans Bengali and Inter; responsive mobile/tablet layouts; keyboard accessibility; readable error, empty and loading states.
- All key workflows should take minimal steps; do not clutter screens with unnecessary charts or animations.

4. Admin information architecture

Route	Main purpose
/admin	Overview, KPIs, alerts, daily closing status
/admin/products	Catalog, categories, SKU, prices, price history
/admin/recipes	Versioned BOM / recipe ingredients and standards
/admin/production	Plan, batch, ingredient usage, actual yield and waste
/admin/inventory	Raw materials, finished goods, lots, counts, transfers
/admin/purchases	Suppliers, orders, receipts, invoices and payments
/admin/sales	Sales history, refunds, payment status
/admin/expenses	Utilities, recurring expenses and allocations
/admin/workforce	Employees, shifts, attendance, payroll
/admin/reports	Sales, production, yield, waste, profit, cash
/admin/settings	Users, permissions, units, categories, audit logs

5. Shop information architecture

Route	Main purpose
/shop	POS product grid, cart, discounts and checkout
/shop/sales	Sales list, receipt reprint, authorized refunds
/shop/stock	Shop availability and received transfers
/shop/waste	Record shop spoilage with reason and approval
/shop/cash	Open/close shift, cash count and differences
/shop/settings	Language, printer and permitted profile settings

6. Master data: products, ingredients and units

- Product: id, SKU, Bangla name, English name, category, unit, optional standard piece weight, selling price, price effective date, image, shelf life, barcode, active flag.
- Ingredient: id, bilingual name, category, base unit, purchase unit, conversion ratio, reorder point, supplier, expiry tracking, storage location.
- Unit conversion must normalize kg/g, liter/ml, piece/packet before calculating recipes or stock; preserve precision.
- Do not hard-delete records referenced by transactions. Archive products and materials instead.
- Create opening stock import and controlled manual stock adjustments with reason, actor and approval.

7. Recipe and production engine

- Recipe versions: product, standard batch size, ingredient quantities, expected output count and total weight, standard waste, effective date.
- Production status: Draft → Planned → In Progress → Completed → Approved; cancellation with authorization and safe reversal rules.
- Batch records: date, shift, operator, recipe snapshot, actual ingredient usage, expected output, actual saleable output, rejected/rework/waste, output weight, labor, allocated utilities, remarks.
- Approval must atomically post ingredient consumption and finished goods output. Prevent double posting via idempotency and status constraints.
- Preserve historical ingredient prices and recipe version snapshots; future recipe edits must not change old costs.

Yield comparison example

Metric	Day 1	Day 2
Ingredient input	10 kg	10 kg
Saleable output	10 pieces	12 pieces
Yield	1.0 pcs/kg	1.2 pcs/kg
Raw ingredient cost (at □50/kg)	□500	□500
Raw input cost / piece	□50	□41.67

Yield improvement by piece count is +20%, but flag changes in piece weight, recipe composition, rejected quantity and quality. Also report output kg/input kg and standard-weight variance. These values are illustrative, not live business data.

8. Inventory and transfers

- Use an immutable inventory movement ledger: purchase receipt, production issue/return, finished production receipt, transfer dispatch/receive, sale, return, waste and adjustment.
- Track raw material lots, expiry, purchase cost and location; track finished goods by production batch, location, quantity and expiry.
- Transfer workflow: Requested → Approved → Dispatched → Received; avoid duplicated stock across locations.
- Use a documented FIFO or weighted-average costing policy consistently for inventory valuation and COGS.
- Low-stock, expiry and negative-stock checks; reconcile stock balances to ledger; never directly overwrite totals.

9. POS, payments and cash

- POS: search, categories, item quantity, available stock, item/cart discount, tax configuration, multiple payment methods, split tender, receipt print.
- Checkout: validate stock and permissions → create sale and sale lines → record payment → deduct stock atomically → issue receipt.
- Use idempotency keys to prevent duplicate sales on retries; preserve unsent cart on network loss without claiming payment success.
- Returns must reference original sale where possible, require reason and authorization, and separately record restockable vs damaged items.
- Cash shift: opening float, cash sales, refunds, paid-out expenses, deposits/withdrawals, expected vs counted closing cash and approval.

10. Purchases, costs, workers and profitability

- Purchase orders, partial receiving, supplier invoice, payment, outstanding balance, purchase return and price history.

- Expenses: rent, gas, electricity, water, transport, cleaning, maintenance, packaging and other configurable categories; attachments and approvals.
- Utilities allocation: batch hours, equipment usage, production quantity or authorized manual method; do not count allocated costs twice.
- Employees: role, contact, join date, shifts, attendance, daily/weekly/monthly pay, overtime, bonus, advance, deduction and payment history.
- Profit: net sales = gross sales − discounts − returns (excluding separately collected taxes); gross profit = net sales − COGS; operating profit = gross profit − operating expenses.
- Separate profit from cash flow. Ingredient purchases increase inventory and liabilities/cash movement; do not immediately expense them again when consumed.

11. Database schema — minimum tables

Identity and configuration

organizations, branches, locations, profiles, roles, permissions, user_role_assignments, system_settings, audit_logs

Products and recipes

products, product_categories, product_prices, raw_materials, units, unit_conversions, recipes, recipe_versions, recipe_ingredients

Production and inventory

production_plans, production_batches, production_inputs, production_outputs, production_waste, inventory_lots, inventory_movements, stock_transfers, stock_transfer_items

Procurement and sales

suppliers, purchases, purchase_items, purchase_receipts, supplier_payments, sales, sale_items, payments, refunds

Operations and finance

waste_records, expense_categories, expenses, utility_allocations, employees, attendance, payroll_periods, payroll_items, payroll_payments, cash_sessions, cash_movements

Every table needs appropriate UUID primary keys, foreign keys, uniqueness, check constraints, created/updated timestamps, branch scoping, indexes and RLS. Use NUMERIC/DECIMAL for money and quantities, not floating-point. Define transactional posting procedures and audit events.

12. Roles, permissions and security

Role	Typical permissions
Owner / Super Admin	Full access, user permissions, financial reports
Manager	Operational oversight and approved adjustments
Production Manager	Recipes, production, batch approval as authorized
Inventory Staff	Stock receiving, counting and transfers
Cashier	POS, assigned register, shop stock, limited returns
Accountant	Expenses, payroll, supplier balances, reports
Viewer	Explicitly granted read-only reports

- Enforce RBAC in server/database with Supabase RLS; menu hiding is not security.
- Cashiers must not read salary, supplier costs, profit margin or admin-only records.
- Never expose service-role keys to browser; validate inputs server-side; protect uploads; audit approvals/refunds/adjustments.
- Provide role-specific tests including unauthorized direct API/database access attempts.

13. Reports and exports

- Daily/weekly/monthly production; planned vs actual output; ingredient usage; batch yield and weight variance; waste and expiry.
- Sales by product/category/shift/payment method; discounts, refunds, cash variance, low stock and supplier balances.
- Raw material cost history, finished goods valuation, COGS, gross margin, operating profit, payroll and utilities.
- Date/product/category/employee/location filters; export Excel, CSV and print-ready PDF; apply role restrictions to exports.

14. Excel migration

- Provide import templates for product master, ingredient master, opening stock, suppliers and employees.
- Upload → preview → validate units/duplicates/required fields → display errors → confirm import → audit import.
- Never auto-import unverified data into production. Demo data belongs only in a separated development environment.

15. Delivery phases and gates

Phase	Deliverable / completion gate
1 — Foundation	App scaffold, authentication, RLS, admin/shop shell, bilingual dictionaries, design tokens; login and permission tests pass.
2 — Master Data	Products, ingredients, units, suppliers, recipes, pricing; CRUD and validation tests pass.
3 — Production	Production plans/batches, material consumption, yield, finished inventory, transfers, waste; ledger reconciliation passes.
4 — Shop	POS, checkout, receipts, returns, cash shifts; duplicate-checkout and stock tests pass.
5 — Operations	Expenses, utilities, employees, attendance, payroll; sample calculations pass.
6 — Analytics	COGS, profit, dashboards, filters and exports; reports reconcile to ledger.
7 — Hardening	Security tests, responsive testing, performance, migrations, deployment and handover.

16. Mandatory acceptance scenarios

☐ Purchase 50 kg flour → raw stock rises exactly 50 kg.

☐ Approve batch consuming 10 kg flour → raw stock falls exactly 10 kg, once only.

☐ Produce 100 saleable units → finished stock rises exactly 100.

☐ Transfer 40 units to shop → warehouse −40, shop +40, total unchanged.

☐ Sell 5 units → shop −5; payment, receipt and COGS posted once.

☐ Record 2 damaged units → stock and waste ledger reconcile.

☐ Edit recipe → historical production cost snapshots unchanged.

☐ Cashier attempts admin financial API → denied by RLS/server.

☐ Switch Bangla/English → navigation, forms, errors, receipts and reports translated.

☐ Retry same checkout request → one sale only.

☐ Close cash shift → expected vs counted cash and variance correct.

☐ Reconcile inventory, sales, COGS and operating profit against source ledgers.

17. Instructions to Google Antigravity — paste as project task

Act as a senior full-stack architect, product designer, database engineer and QA lead. Implement this PDF as the authoritative product specification. First inspect the existing repository and identify reusable working code. Produce a concise ERD, route map, permission matrix and implementation checklist. Then build phases sequentially, preserving existing functionality. Never replace functional modules with static mockups. Use real database-backed workflows, transactional stock and payment posting, bilingual UI, secure RBAC/RLS, and financial calculations that reconcile. After each phase run lint, typecheck, relevant automated tests and production build; fix failures before proceeding. Maintain a PROGRESS.md file listing complete, pending and blocked items with evidence. Do not claim production readiness until all acceptance scenarios pass.

Final deliverables

- Complete source code, working /admin and /shop portals, bilingual localization and mobile-responsive UI.
- Supabase SQL migrations, RLS policies, secure auth, database transactions and audit logs.
- Functional production, inventory, POS, cash management, costing, payroll, reporting and exports.
- README, .env.example, deployment guide, import templates, automated tests and verified production build.

Implementation principle: Simple enough for a new cashier to use confidently; rigorous enough for an owner to trust stock, production cost and profit numbers.